AUTOMATIC SPEECH RECOGNITION FOR EMOTION DETECTION USING MACHINE LEARNING

THEME: Design and implementation of a Machine Learning algortihm capable of classifying 8 emotional states from audio files

STUDENT: BLAGA MARIA

GROUP: 2331

Tabel of contents:

1. Introduction ..... page 2
2. Theoretical Background ..... page 3
3. Software Implementation ..... page 4
4. Conclusion ..... page 9
5. Bibliography ..... page 10

1. INTRODUCTION

The primary requierement of this project is to develop a system that can hear and interpret human emotion, going beyond simple speech-to-text recognition. For this project, the objective was to design and develop a system capable of analyzing the acoustic properties of human speech and classifying the emotional state into one of eight distinct categories: Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, and Surprised.

The system must be able to process raw audio data and provide a reliable prediction based on learned patterns. The problem was addressed using a Machine Learning pipline based on a Multi-Layer Perceptron(MLP) neural network. The process involved:

- Data Acquisition: utilizing the RAVDESS dataset, which provides balanced emotional samples from professional actors;

- Feature Extraction: using the librosa library to extract Mel-Frequency Cepstral Coeffcients (MFCC) and Mel-Spectograms, which serve as the acoustin fingerprint of the voice.

- Model Training: configuring a deep neural network with three hidden layers to achieve a good accuracy.

A significant new aspect of this implementation is the ingration of an interactive Graphical User Interface built with Gradio. This interface allow for real time interaction, where a user can upload a file and receive an instantaneous emotional analysis, making the technology accessible and easy to demonstrate.

2. THEORETICAL BACKGROUND

The recognition of emotions from speech involves complex signal processing and pattern recognition. This is the technology that allows a computer to understand how something is said, not just what is said. The system works in three simple steps:

1. The data source: RAVDESS Dataset

   To train a computer, we need examples. We used RAVDESS dataset, which is a collection of audio files recored by a professional actors. Each file name tells the computer exactly what emotion is inside (ex: „ 03” means Happy). By listening to hundreds of these examples, the AI learns the difference between the sound of anger and the sound of calm.

2. Creating the „Voice fingerprint”

   A computer cannot hear voices like we do, it only sees numbers, so we use a library called Librosa to transform audio into a fingerprint:

   - MFCC (The voice shape): This captures the power of the voice at different frequencies. It focuses on what the human ear actually hears, ignoring background noise.
   - Mel-Spectogram(The color of sound): This turns into a visual map of frequencies. It helps the AI see if a voice is high-pitched(like in Surprised) or low and slow (like in Sad).
   - The brain (MLP Neural Network)

     The Multi-Layer Perceptron is the AI brain of our project. It is ogranized in layer of neurons:

     - The input layer: takes the voice fingerprint numbers
     - The hidden layer: this is where the thinking happens. We used a deep structure(1024-512-256 neurons) to help the model recognize very subtle emotional changes.
     - The output layer: give the final answer.

The AI listens to a clip and makes a guess, if it guess is wrong, the system calculates the error, using a optimizer, the AI slightly changes its internal connections so that next time it won’t make the same mistakes.

3. SOFTWWARE IMPLEMENTATION

The implementation of the Speech Emotion Recognition system was carried out using Python. The project is structured into three main stages: Data Preprocessing, Model Training, and the Development of the Interactive Interface.

The first step of the implementation involves importing the necessary libraries for audio processing and machine learning. Each library has a specific role in the pipeline:

```python
# Importul bibliotecilor esențiale pentru ML și analiza audio

import librosa        # Procesarea semnalului audio (MFCC)
import soundfile      # Citirea fișierelor audio
import numpy as np    # Operații matematice
import glob           # Pentru a găsi toate fișierele audio în foldere
import os             # Pentru lucrul cu căi de fișiere

# Biblioteci pentru Machine Learning
from sklearn.model_selection import train_test_split # Split în train/test
from sklearn.neural_network import MLPClassifier     # Modelul de rețea neuronală
from sklearn.metrics import accuracy_score           # Evaluarea performanței
```

Librosa: This is the most critical library for our project as it allows us to load .wav files and convert them into MFCCs (Mel-Frequency Cepstral Coefficients). MLPClassifier: This represents the intelligence of the system. It is a robust neural network capable of learning non-linear patterns in voice data.

Then we have, audio feature extraction function. This section describes the core function used to transform raw .wav files into a compact numerical vector. The function is designed to handle potential errors and ensure that only valid audio signals are processed.

```python
# Functia de extragere a caracteisticilor MFFC si Mel-Spectogram din fisierele audio
def extract_feature(file_name, mfcc=True, mel=True):
    try:
        with soundfile.SoundFile(file_name) as sound_file:
            X = sound_file.read(dtype="float32")
            sample_rate = sound_file.samplerate

            # Verificăm dacă fișierul este prea scurt
            if len(X) < 2048:
                return None

            result = np.array([])

            # 1. MFCC - Rămâne activ
            if mfcc:
                mfccs = np.mean(librosa.feature.mfcc(y=X, sr=sample_rate, n_mfcc=40).T, axis=0)
                result = np.hstack((result, mfccs))

            # 2. Mel Spectrogram - Rămâne activ
            if mel:
                mel_spec = np.mean(librosa.feature.melspectrogram(y=X, sr=sample_rate).T, axis=0)
                result = np.hstack((result, mel_spec))

            return result
    except Exception as e:
        return None
```

The extract_feature function is the core of the preprocessing pipeline, serving two main purposes:

- Acoustic Filtering: It isolates MFCCs and Mel-Spectrograms, which act as the acoustic fingerprint of the voice. This allows the model to focus on pitch and spectral intensity the most relevant features for human emotion.

- Data Uniformity (np.mean and np.hstack): Since audio clips have different lengths, we use np.mean to create a fixed-size input for the neural network, preventing system crashes. Using np.hstack, we glue different types of audio data (tone and energy) into a single, enriched input for better accuracy.

The next module is responsible for scanning the dataset directory and associating each audio file with its corresponding emotion. The implementation follows the specific naming convention of the RAVDESS dataset.

```python
import os
import glob
import numpy as np
from sklearn.model_selection import train_test_split

#  DEFINIREA EMOȚIILOR
emotions = {
    '01':'neutral',
    '02':'calm',
    '03':'happy',
    '04':'sad',
    '05':'angry',
    '06':'fearful',
    '07':'disgust',
    '08':'surprised'
}

# Aici alegem emotiile
observed_emotions = ['neutral', 'calm', 'happy', 'sad', 'angry', 'fearful', 'disgust', 'surprised']

#  FUNCȚIA DE ÎNCĂRCARE DATE
def load_data(test_size=0.25):
    X, y = [], []
    count = 0

    # Construim calea pentru Windows
    cale_cautare = os.path.join("actors_voices", "Actor_*", "*.wav")
    files = glob.glob(cale_cautare)
```

This section prepares the data for the model:
Emotion Dictionary: Maps the numerical codes from RAVDESS filenames (e.g., '01', '03') to actual labels like neutral or happy.
Target Selection: Defines the observed_emotions list to filter and focus only on the 8 specific categories chosen for this project.

```python
    for file in files:
        file_name = os.path.basename(file)
        parts = file_name.split("-")

        if len(parts) >= 3:
            emotion_code = parts[2]

            # Verificăm dacă codul există în dicționarul nostru
            if emotion_code in emotions:
                emotion = emotions[emotion_code]

                # Verificăm dacă emoția este una din cele 8 pe care le vrem
                if emotion in observed_emotions:
                    # Folosim funcția de extracție creată anterior
                    feature = extract_feature(file, mfcc=True, mel=True)
                    if feature is not None:
                        X.append(feature)
                        y.append(emotion)
                        count += 1

    print(f" Am procesat cu succes {count} fișiere pentru cele 8 emoții alese. ")

    if count == 0:
        print("EROARE: Nu am putut procesa niciun fișier. Verifică folderul 'actors_voices'!")
        return None, None, None, None

    return train_test_split(np.array(X), y, test_size=test_size, random_state=9)

#  PASUL C: RULARE
X_train, X_test, y_train, y_test = load_data(test_size=0.25)
```

This section executes the loop that processes each audio file found in the dataset: The code splits the filename to find the emotion_code (the 3rd element). This allows the system to identify what emotion the actor was recording. It checks if the emotion matches our list and then calls the extract_feature function to turn the audio into a fingerprint. If a file is valid, it is added to our dataset.

Data Splitting Strategy: We split the data into Training (75%) and Testing (25%) sets. The Training set is used to teach the model patterns, while the Testing set acts as an exam with new, unseen data.

For model training and performance we have next cell:

```python
# Model pentru 8 emoții
model = MLPClassifier(
    alpha=0.001,                # Penalizare mai mică (permite modelului să fie mai complex)
    batch_size=128,             # Procesăm datele în bucati mai mici
    hidden_layer_sizes=(1024, 512, 256), # Trei straturi
    learning_rate='adaptive',
    max_iter=1000,              # Mai multe încercări de învățare
    random_state=9,
    verbose=True                # Aceasta ne va arăta progresul în timp real
)

model.fit(X_train, y_train)

# Evaluare
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_true=y_test, y_pred=y_pred)
print("\n" + "="*40)
print(f"ACURATEȚE: {accuracy*100:.2f}%")
print("="*40)
```

The model uses a deep MLP (Multi-Layer Perceptron) architecture with three layers of thinking to classify the 8 emotional states. By monitoring the Loss values during training, we can see the model improving its predictions over time. The final accuracy of 61.00% demonstrates a high capability of identifying voice characteristics in human speech compared to a random baseline.

```
==================================
ACURATEȚE: 61.00%
==================================
```

```python
# Functia de recunoastere a emotiei
def recunoaste_emotia(cale_fisier):
    # 1. Aplicăm procesarea pe fișierul nou
    caracteristici = extract_feature(cale_fisier, mfcc=True, mel=True)

    if caracteristici is not None:
        # 2. Pregătim datele pentru model
        caracteristici = caracteristici.reshape(1, -1)

        # 3. Întrebăm creierul antrenat ce crede
        predictie = model.predict(caracteristici)

        return predictie[0]
    else:
        return "Nu am putut procesa fisierul."

# TESTARE

fisier_nou = "actors_voices/Actor_05/03-01-03-01-01-01-05.wav"

rezultat = recunoaste_emotia(fisier_nou)

print(f" The emotion is: {rezultat.upper()}")
```

This final part of the code represents the practical application of our trained model. It describes how the system processes a completely new audio file to predict the emotion.

Before the model can listen to a new file, the audio must go through the same extraction process as the training data (MFCC and Mel-Spectrogram). The model expects a specific format. Since we are testing only one file at a time, we tell the computer to treat the data as a single row.

We ask our trained AI brain to look at the features and decide which of the 8 emotions fits best. The system returns the predicted label providing a clear, human-readable answer.In our code, we tested a specific file from "Actor_05". The system analyzed the acoustic patterns and correctly identified the emotion, demonstrating that the training was successful.

The final stage of the implementation is the development of a Graphical User Interface (GUI) using the Gradio library. This interface allows any user to interact with the trained AI model without needing to write or understand code.

```python
import gradio as gr

def predict_emotion_ui(audio_file):
    try:
        # Folosim doar ce am antrenat: mfcc=True, mel=True

        features = extract_feature(audio_file, mfcc=True, mel=True)

        if features is not None:
            features = features.reshape(1, -1)
            prediction = model.predict(features)
            emotia = prediction[0]

            # Mapare vizuală
            emoji_map = {
                "happy": "😊 Happy", "sad": "😢 Sad", "angry": "😡 Angry",
                "calm": "😌 Calm", "neutral": "😐 Neutral", "surprised": "😲 Surprised",
                "fearful": "😨 Fearful", "disgust": "🤢 Disgust"
            }
            return emoji_map.get(emotia, emotia.upper())
        else:
            return " Could not process sound."
    except Exception as e:
        return f" Eroare: {str(e)}"
```

Users can upload or drag-and-drop a .wav file into the Audio Input box. To make the results user-friendly, the system uses an emoji map that connects the raw text prediction to a visual icon. When the Detect Emotion button is clicked, the system instantly processes the sound and displays the detected emotion on the screen.

```python
# Interfață curată
with gr.Blocks(title="AI Emotion Detector") as demo:
    gr.Markdown("#  Voice Emotion Analyzer")
    gr.Markdown(f"Trained model")

    with gr.Row():
        with gr.Column():
            audio_input = gr.Audio(type="filepath", label=" Load a .wav file from the dataset")
            submit_btn = gr.Button("🔍 Detect Emotion", variant="primary")
        with gr.Column():
            output_text = gr.Label(label="Result")

    submit_btn.click(fn=predict_emotion_ui, inputs=audio_input, outputs=output_text)

demo.launch()
```

This section creates the functional link between the user and the AI model. It defines the Audio Input for uploading files, the Action Button to trigger the analysis, and the Result Label to display the detected emotion. The click function ensures that the audio data is correctly sent to the neural network and the response is shown to the user in real-time.

4. CONCLUSION

The objective of this project was to develop a system capable of identifying human emotions through voice analysis. By integrating audio processing techniques with machine learning, the following results were achieved:

- Effective Feature Extraction: The use of MFCC and Mel-Spectrograms proved to be a reliable method for converting raw audio into a "vocal fingerprint" that the computer can understand.
- Solid Performance: The MLP (Multi-Layer Perceptron) model reached an accuracy of 61%. While recognizing 8 different emotions is a difficult task, this result shows that the system can successfully identify key patterns, especially in high-energy emotions like anger or happiness.
- Practical Accessibility: The implementation of the Gradio interface demonstrated that the model is not just a theoretical tool, but a functional application that provides instant, user-friendly feedback.

This project serves as a strong foundation for Speech Emotion Recognition. Future improvements could include expanding the dataset or exploring more complex neural network architectures to further refine the accuracy of the detection.

5. BIBLIOGRAPHY

https://www.educative.io/blog/speech-emotion-recognition

https://data-flair.training/blogs/python-mini-project-speech-emotion-recognition/

https://www.kaggle.com/datasets/uwrfkaggler/ravdess-emotional-speech-audio